



Universidad Veracruzana

Facultad de Negocios y Tecnologías

Licenciatura en Ingeniería de Software

Experiencia educativa:
Principios de Diseño de Software

Alumno:
Víctor Eduardo Córdoba Bravo

Documentación del Proyecto

PhonePedia

Docente: Adolfo Centeno Tellez

Periodo: Periodo Feb-Jul 2026

Ixtaczoquitlán, Veracruz

mayo de 2026

1. Introducción

PhonePedia es un proyecto de código libre que permite a las personas compartir información acerca del mundo de la telefonía móvil, concentrándose en publicaciones informativas acerca de marcas de celulares y teléfonos pertenecientes a dichas marcas.

El proyecto se centra principalmente en una aplicación móvil que permite al usuario crear una cuenta y realizar publicaciones de celulares o marcas de teléfonos en la industria, compartiendo especificaciones técnicas e informativas para aquellos usuarios entusiastas de la tecnología móvil.

Adicionalmente, el desarrollo de PhonePedia se enfoca ampliamente en la seguridad del usuario, tanto como para su cuenta como para sus publicaciones, impidiendo a usuarios maliciosos modificar registros o robar credenciales de otros usuarios.

Por otro lado, el desarrollo a nivel técnico es experimental, destacando la implementación de patrones de diseño estructurales y creacionales, para asegurar un código legible, mantenible y testeable. Durante la etapa de codificación, se aseguró la implementación de patrones de diseño que pudieran ser útiles y que aporten un valor real al proyecto en rendimiento y mantenibilidad.

Finalmente, la arquitectura del proyecto PhonePedia está pensada para ser segura, siendo dividida en tres capas:

1. Capa de persistencia de datos o base de datos.
2. Capa de lógica de negocio o API.
3. Capa de aplicación móvil.

Con esta marcada separación de responsabilidades dentro del sistema, se asegura la integridad de la capa de persistencia de datos, siendo solo accesible mediante la API que conecta la aplicación móvil y la base de datos. De esta forma, se garantiza el acceso controlado a la capa de persistencia de datos.

2. Estructura del Proyecto

Como se definió en la introducción, la arquitectura del proyecto está dividida en tres capas con solamente una responsabilidad, siguiendo el principio Separation of Concerns (SoC). Asimismo, cada capa fue desarrollada con una tecnología adaptable al proyecto, lo que implica que el proyecto está desarrollado con las tecnologías que mejor compensan las necesidades en el desarrollo de PhonePedia.

A continuación, se presenta cada capa del sistema, junto a su objetivo y stack tecnológico.

2.1. Capa de Aplicación Móvil

En esta capa, se encuentra el proyecto realiado en flutter que representa la capa de presentación al cliente final. En otras palabras, aquí se encuentra el código de la aplicación móvil con la que los usuarios entusiastas de la tecnología móvil podrán interactuar entre sí.

La tecnología utilizada para esta capa del proyecto es **flutter**, siendo un framework con todas las herramientas necesarias para un desarrollo rápido y preciso, con una estructura de alta calidad, con métodos profesionales de depuración y compilación.

Adicionalmente, se ha elegido esta tecnología gracias a su alta flexibilidad y adaptabilidad en desarrollo de aplicaciones móviles. Asimismo, las interfaces modernas con íconos y botones pre-programados para ser funcionales y elegantes destacan sobre otros frameworks de desarrollo móvil.

Por último, cabe destacar que en esta capa del proyecto es donde se aplicó la refactorización implementando patrones de diseño que nos aportan funcionalidad, eficiencia y mejora de lectura de código, para adaptarnos a estándares de calidad de código profesionales. Por lo tanto, la descripción de la capa de aplicación móvil nos ayudará más adelante en el documento para explicar la implementación de patrones de diseño dentro del código.

2.2. Capa de Lógica de Negocio

La capa de lógica de negocio es donde se encuentra viviendo la **API** del sistema. Application Programming Interface por sus siglas, es el puente lógico entre la aplicación de móvil y la base de datos. Es la encargada de traducir las peticiones provenientes de la app de flutter a instrucciones que nuestro motor de base de datos pueda entender.

Asimismo, en esta capa es donde se encuentra la lógica de negocio de nuestra aplicación móvil, lo que significa que es aquí donde definimos las reglas que el sistema debe de manejar. A continuación, presento algunos ejemplos de reglas de negocio que debemos de manejar en nuestra aplicación.

1. Un usuario NO puede modificar las publicaciones de otros usuarios.
2. Un usuario solo puede modificar sus propias publicaciones.
3. Un usuario NO puede tener más de una reacción en un ítem reaccionable (publicaciones de teléfonos o comentarios).
4. Una vez publicado un comentario, NO se puede borrar.
5. Un usuario NO puede modificar las reacciones de otros usuarios.

Las lógica necesaria para cumplir con las reglas de negocio recientemente definidas es desarrollada dentro de la capa de Lógica de Negocio, específicamente en la **API** del sistema.

Esto es, esta capa tiene la responsabilidad y razón de cambio de cumplir con las validaciones necesarias para satisfacer las necesidades y reglas del negocio.

Finalmente, la tecnología utilizada debido a su gran rendimiento y estructura profesional es el framework **Spring Boot**. Se ha elegido esta tecnología debido a su gran adaptabilidad y rapidez al momento de crear un nuevo proyecto. Asimismo, su estructura profesional nos permite adaptar el código a las necesidades del sistema, al igual que no permite tener un código perfectamente mantenible y escalable.

2.3. Capa de Persistencia de Datos

La capa de persistencia de datos es sumamente importante para mantener la integridad y confiabilidad de los datos de sistema. Trabajar en una base de datos segura nos ayuda a garantizar la correctitud de los datos para poder utilizarlos en la aplicación móvil.

En el caso de esta capa, se ha elegido el motor de base de datos **PostgreSQL** desplegada en una página web llamada **Aiven** que nos ofrece servicios de despliegue gratuitos para proyectos experimentales pequeños, lo cual es más que suficiente para las necesidades reales de este proyecto.

Se utiliza una base de datos desplegada en la nube por razones de futura escalabilidad del proyecto, puesto que, una posible necesidad de crecimiento exponencial en las publicaciones en la aplicación móvil sería totalmente satisfecha teniendo un servicio de base de datos desplegado en la nube. Asimismo, este despliegue garantiza la seguridad y disponibilidad de los datos al ser accesibles sin necesidad de tener un servidor personal en un lugar cercano de la zona de despliegue.

3. Patrones de Diseño

Como se describió en la introducción, con fines experimentales y educativos, se implementaron los patrones de diseño que pudieran haber sido benéficos para el desarrollo del proyecto a nivel de código.

Un patrón de diseño es una **solución reutilizable** y estandarizada para resolver problemas comunes en el desarrollo de software. Funciona como un **plano flexible** o una guía que los programadores adaptan a sus necesidades, lo que permite ahorrar tiempo, evitar errores y escribir código legible, mantenible y testeable.

3.1. Contexto de la implementación.

Como se comentó con anterioridad, para fines educativos se evaluaron las posibles ventajas y desventajas que traería la implementación de patrones de diseño *creacionales y estructurales*, con el objetivo de conocer su importancia, utilidad y forma de adaptarlos a proyectos de software.

A continuación, se comparte la definición de un patrón creacional y estructural.

- **Creacionales:** Se enfocan en los mecanismos de creación de objetos. Ayudan a crear sistemas independientes de cómo se inicializan y componen sus objetos. Por ejemplo: singleton o Factory Method).
- **Estructurales:** Tratan sobre la composición de clases y objetos. Facilitan el diseño al asegurar que si una parte del sistema cambia, no es necesario modificar todo el conjunto. Por ejemplo: Decorator o Adapter.

Partiendo de estas definiciones, a continuación se presenta el análisis de la posible inclusión de cada uno de los patrones creacionales y estructurales, con el fin de tener una tabla para llevar la trazabilidad de qué patrones ya se han implementado.

3.2. Patrones Creacionales

Los patrones creacionales se enfocan principalmente en la creación de objetos. La inclusión de patrones creacionales dentro del código del proyecto garantizarían el bajo acoplamiento dentro de las clases principales del sistema, por lo tanto, implementarlos sería potencialmente benéfico.

En total, existen 5 patrones de diseño creacionales. A continuación, se presenta el análisis del valor que aportaría cada uno de los 5 patrones, y si se determinó que sería útil su implementación o no.

3.2.1. Singleton

El patrón diseño creacional *singleton* se enfoca en garantizar que habrá una única instancia global de un objeto en todo el sistema.

Dentro del proyecto, sería de gran utilidad con la conexión con la API del sistema puesto que garantizaría que toda la aplicación use el mismo cliente *HTTP* y comparta la misma configuración de red. Esto significa que por cada usuario que utilice la aplicación habría una única conexión con la API, lo que aporta significativamente en el rendimiento del sistema completo.

Por otro lado, tener una única conexión a internet aporta en la eficiencia de la aplicación dentro del sistema operativo del cliente, debido al ahorro de memoria al evitar múltiples conexiones que resultan redundantes al servidor en internet.

Debido a estas razones, se ha determinado que la inclusión del patrón de diseño singleton aportaría significativamente, por lo tanto, se incluirá dentro del desarrollo.

3.2.2. Factory Method

Factory Method es un patrón creacional que aporta a manejar de mejor forma el acoplamiento de las clases que conviven dentro del sistema. Una clase factory method está hecha

con el objetivo de llevar a cabo la creación de un objeto con todas sus configuraciones y listo para usar. De esta forma, evitamos tener creaciones crudas con la palabra *new* dentro de las clases pertenecientes a la vista para el usuario.

Dentro del proyecto, implementar el factory method en creaciones de objetos como un *ErrorSnackBar* o *SuccessSnackBar* nos evitaría tener código duplicado dentro de nuestras clases. Implementándolo en estos dos ejempls, estandarizaría el diseño de las alertas en toda la aplicación y haría que, si en el día de mañana se necesita cambiar el color de los errores a naranja, solo se deba modificar una línea de código en la fábrica.

Por estas razones, la implementación del patrón factory method dentro del proyecto, sería de gran ayuda para la refactorización del código, cumpliendo con el *principio de responsabilidad única* puesto que, la clase *snackbar factory* sería encargada de únicamente la creación de mensajes de alerta dentro de la aplicación.

3.2.3. Abstract Factory Method

El patrón Abstract Factory Method proporciona una interfaz centralizada en crear familias de objetos relacionados o dependientes de otros, sin especificar sus clases concretas. Como su hermano creacional Factory Method, **promueve el bajo acoplamiento** en la lógica de creación en el código fuente de la interfaz gráfica, permitiendo que el sistema sea independiente de la implementación específica de dichos objetos.

Dentro del proyecto, se podría implementar un Abstract Factory Method en la **creación de elementos gráficos**, tales como inputs, botones o modales. Al implementar este patrón, agrupamos la creación de estos componentes visuales en una "fábrica de familia de componentes".

De esta forma, si en el día de mañana queremos expandir la aplicación móvil al mercado de *iOS*, no tendríamos que modificar la creación de todos y cada uno de los elementos gráficos dentro del código de la aplicación, sino que, tendríamos que crear una fábrica distinta preparada exactamente para la creación de componentes adaptados al sistema operativo *iOS*.

Por estas razones, se ha determinado que la inclusión del patrón Abstract Factory Method dentro del proyecto será aceptada, más específicamente en los archivos *app ui factory* con su implementación concreta *material form factory*.

3.2.4. Builder

El patrón Builder es un patrón de diseño creacional que separa la construcción de un objeto complejo, permitiendo construirlo paso a paso. Es especialmente útil cuando un objeto requiere una gran cantidad de parámetros, o cuando la validación y transformación de los datos ensucia la lógica del cliente.

Implementar el patrón Builder dentro del sistema sería altamente beneficioso, puesto que

debido a la naturaleza del sistema (celulares con una gran cantidad de especificaciones técnicas), resulta en un gran acoplamiento en la capa de la interfaz de usuario la instanciación de un objeto de la clase teléfono con un total de nueve argumentos.

Al crear un *TelefonoBuilder*, sacamos toda la lógica del parseo y construcción fuera del archivo principal, pues la vista solamente le pasaría los textos crudos al constructor y el Builder se encargaría de validarlos, transformarlos y finalmente, instanciar el objeto.

Dadas estas razones, resulta benéfico la implementación del patrón builder dentro de los archivos *telefono builder dart* dentro del proyecto.

3.2.5. Prototype

Prototype permite copiar o clonar objetos existentes sin hacer que el código dependa de sus clases concretas. Este patrón promueve la inmutabilidad, debido a que, en lugar de modificar los atributos de un objeto existente en memoria, se crea una copia exacta de su objeto original, alterando únicamente los campos que se necesitan cambiar, manteniendo el resto intacto.

Dentro del contexto del proyecto, implementar el patrón Prototype podría aportar significativamente a la experiencia de usuario de la aplicación. Implementar el patrón al momento de que un usuario reacciona a una publicación de un teléfono, se podría clonar el objeto teléfono en memoria y sumarle una reacción al contador en la copia. De esta forma, la interfaz se actualizaría al instante, mientras que por otro lado, la petición a la API ocurriría en segundo plano.

Dada esta razón, se ha determinado que la inclusión del patrón Prototype dentro del código del proyecto sería altamente beneficioso para una mejor experiencia de usuario. Más específicamente, se implementaría en la clase *telefono.dart* y en el *main.dart*.

3.3. Patrones Estructurales

Los patrones estructurales tratan principalmente sobre la ensamblación de clases y objetos. Facilitan el diseño al asegurar que si una parte del sistema cambia, no es necesario modificar todo el conjunto, aportando al cumplimiento con el principio de inversión de dependencias.

En total, existen 6 patrones de diseño estructurales. A continuación, se presenta el análisis del valor que aportaría cada uno de los 6 patrones, y si se determinó que sería útil su implementación o no.

3.3.1. Adapter

Adapter es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles. Actúa como un intermediario que transforma la interfaz de una

clase en otra que el cliente espera. Aporta al cumplimiento del principio de responsabilidad única al separar la lógica de conversión de datos con la interfaz de usuario.

En el contexto del proyecto, el patrón adapter sería de gran utilidad para el almacenamiento de fechas de salida de los teléfonos. Crear un **DateAdapter** ayudaría a tomar fechas crudas e incompatibles para adaptarlas a un formato amigable y legible para el usuario. De esta forma, la interfaz gráfica no se ensuciaría haciendo conversiones de fechas.

Dada esta razón, se ha determinado que la inclusión del patrón adapter tendría un gran valor para la experiencia de usuario, por lo tanto, se incluirá en el desarrollo del proyecto con la clase *date adapter*.

3.3.2. Bridge

El patrón Bridge es un patrón de diseño estructural que divide una clase grande o un grupo de clases estrechamente relacionadas en dos jerarquías separadas.

- **Abstracción:** La capa de control de alto nivel (Interfaces, clases abstractas).
- **Implementación:** La capa de plataforma o ejecución de bajo nivel.

Estas dos jerarquías pueden desarrollarse y evolucionar de forma independiente. Bridge fomenta el uso de la composición sobre la herencia.

En el contexto del proyecto, utilizar el patrón Bridge sería potencialmente útil para cumplir con el principio de inversión de dependencias. Crear un *AppCatalogBridge* haría que la interfaz gráfica dependa únicamente de su abstracción, haciendo que si en un futuro se cambia la base de datos, simple y sencillamente se implemente el contrato de *IApiRepository*, logrando que en el main no cambie absolutamente nada.

Dada esta razón, la implementación del patrón Builder será llevada a cabo con el archivo *app catalog bridge*.

3.3.3. Decorator

El patrón Decorator permite añadir nuevas funcionalidades o comportamientos a los objetos de forma dinámica colocándolos dentro de objetos encapsuladores especiales. Proporciona una alternativa flexible a la herencia para extender la funcionalidad de una clase, cumpliendo a la perfección con el principio de *Open/Closed Principle*.

En el contexto del proyecto y orientado a un desarrollo limpio con errores trazables, se ha ideado la inclusión del patrón Decorator con un *logger*. Con este patrón, se creará un *ApiLoggerDecorator*, que envolverá al servicio. El decorador medirá el tiempo, hará las impresiones en consola para llevar la trazabilidad de los registros con la API en el *ApiService*.

Por esta razón, Decorator se llevará a cabo en el archivo *api logger decorator*.

3.3.4. Facade

El patrón Facade es un patrón de diseño estructural que proporciona una interfaz simplificada con bajo acoplamiento para un subsistema complejo de clases, librerías o frameworks. Su objetivo principal es ocultar la complejidad estructural y reducir el acoplamiento entre el cliente y las múltiples clases con las que de otro modo tendría que interactuar.

En el contexto del proyecto, crear una clase que implemente el patrón Facade para dibujar la interfaz gráfica, sería altamente beneficioso para un código limpio y con separación de responsabilidades. Un posible *DashboardFacade* tendría un solo método simple: *getFullDashboard()*. Este método se encargaría de hacer múltiples llamadas, empaquetar los datos y entregarle a la UI un solo objeto limpio y listo para dibujarse en la pantalla del usuario.

3.3.5. Flyweight

El patrón Flyweight es un patrón de diseño estructural enfocado en la optimización de recursos. Permite mantener una gran cantidad de objetos en la RAM, compartiendo las partes comunes entre varios objetos, en lugar de mantener toda la información en cada uno de ellos. Divide el estado del objeto en dos:

- **Intrínseco:** constante y compartido.
- **Extrínseco:** Variable y único por contexto.

En el contexto del proyecto, crear clases *enum* resulta altamente eficiente debido a la minimización de uso de memoria RAM, al compartir la mayor cantidad de datos posibles con objetos similares.

La clase ENUM son una implementación perfecta de este patrón, ya que garantizan que las instancias sean constantes en tiempo de compilación y compartidas globalmente a lo largo de toda la aplicación. En el proyecto, la clase ENUM es implementada en el archivo *reaccion tipo*.

3.3.6. Proxy

El patrón Proxy es un patrón de diseño estructural que proporciona un objeto sustituto de otro objeto para controlar el acceso a este. El proxy intercepta las llamadas al objeto real, lo que permite realizar tareas previas o posteriores a la petición.

En el contexto del proyecto, implementar un *ApiAuthProxy* sería de gran utilidad para garantizar que cada una de las peticiones que se hagan al servidor de la API valgan la pena en términos de los recursos del servidor y tiempo de CPU en el celular del cliente. La idea es que, este proxy detecte antes de enviar una petición que esta contenga en sus cabeceras su token de autenticación, y si no hay token, el proxy bloqueará la petición y lanzará un error local internamente, protegiendo a la red y al servidor de recibir peticiones inútiles.

Dada esta razón, se creará el archivo *api auth proxy* para la implementación del patrón proxy dentro del proyecto.

3.4. Tabla de Patrones a implementar

Cuadro 1: Resumen de Implementación de Patrones de Diseño

Tipo de Patrón	Nombre	Integración	Archivo
Creacional	Singleton	Sí	services/api_service.dart
Creacional	Factory Method	Sí	factories/snackbar_factory.dart
Creacional	Abstract Factory	Sí	factories/ui/app_ui_factory.dart factories/ui/material_form_factory.dart
Creacional	Builder	Sí	builders/telefono_builder.dart
Creacional	Prototype	Sí	models/telefono.dart
Estructural	Adapter	Sí	adapters/date_adapter.dart
Estructural	Bridge	Sí	bridges/app_catalog_bridge.dart
Estructural	Composite	No	N/A
Estructural	Decorator	Sí	decorators/api_logger_decorator.dart
Estructural	Facade	Sí	facades/dashboard_facade.dart
Estructural	Flyweight	Sí	models/reaccion_tipo.dart
Estructural	Proxy	Sí	proxies/api_auth_proxy.dart

4. Implentación a nivel de código

Una vez que se ha definido qué patrones se implementarán en el desarrollo del proyecto, a continuación, se presenta la evidencia/código que se desarrolló para cumplir con la implementación de en total, los 10 patrones de diseño.

Es importante recordar que, donde haremos la implementación de los patrones de diseño es en el proyecto de **flutter**, donde vive la aplicación de teléfono con la que el usuario final interactúa ya sea desde creando publicaciones de teléfonos o marcas y reaccionando o comentando en dichas publicaciones para compartir opiniones

Asimismo, es importante destacar que el lenguaje de programación que utiliza nativamente flutter es **dart**, por lo tanto, los archivos de código que se necesiten crear para implementar los patrones tienen la extensión **.dart**.

4.1. Implementación de Patrones Creacionales

4.1.1. Implementación de Singleton

A continuación, se presenta la implementación del patrón Singleton en Dart.

```
lib/services/api service.dart
```

```
class ApiService implements IApiRepository {
  // 1. Instancia estática y privada: Almacena en memoria la única copia
  // permitida de esta clase.
  static final ApiService _instance = ApiService._internal();

  final String baseUrl = 'https://proyecto-apilogin.onrender.com/api';
  late http.Client _httpClient;

  // 2. Constructor Privado (_internal): En Dart, usar un guion bajo hace que
  // el constructor sea privado. Esto bloquea la posibilidad de instanciar
  // la clase desde otros archivos usando 'ApiService()'.
  ApiService._internal() {
    _httpClient = http.Client();
  }

  // 3. Constructor Factory: Actúa como un interceptor. Cuando cualquier parte
  // de la app intenta hacer 'final api = ApiService();', Dart llama a este
  // factory y devuelve la instancia global ya existente en lugar de crear una nueva.
  factory ApiService() {
    return _instance;
  }

  // 4. Método de acceso estático (Alternativa clásica al Factory): Permite
  // obtener la instancia llamando explícitamente a 'ApiService.getInstance()'.
  static ApiService getInstance() {
    return _instance;
  }
}
```

4.1.2. Implementación de Factory Method

A continuación, se presenta la implementación del patrón Factory Method en Dart.

lib/factories/snackbar factory dart

```
class SnackBarFactory {
  static SnackBar create({
    required AlertType type,
    required String message,
  }) {
    Color backgroundColor;
    IconData icon;

    // La fábrica decide la configuración basándose en el tipo
    solicitado
    switch (type) {
      case AlertType.success:
        backgroundColor = Colors.green.shade600;
        icon = Icons.check_circle_outline;
        break;
      case AlertType.error:
        backgroundColor = Colors.red.shade600;
        icon = Icons.error_outline;
        break;
    }

    // Retornamos el objeto final ya construido
    return SnackBar(
      content: Row(
        children: [
          Icon(icon, color: Colors.white),
          const SizedBox(width: 12),
          Expanded(
            child: Text(
              message,
              style: const TextStyle(color: Colors.white,
                fontWeight: FontWeight.bold),
            ),
          ),
        ],
      ),
    );
  }
}
```

4.1.3. Implementación de Abstract Factory Method

A continuación, se presenta la implementación del patrón Abstract Factory Method en Dart.

```
lib/factories/ui/app ui factory.dart
```

```
// INTERFAZ
abstract class IAppUIFactory {
  // Define la creación de un campo de entrada de texto
  Widget createInput(
    TextEditingController controller,
    String label, {
    String? hint,
    TextInputType? keyboardType,
    int maxLines = 1,
  });

  // Define la creación del botón principal de acción
  Widget createPrimaryButton(
    BuildContext context,
    String text,
    VoidCallback onPressed,
  );

  // Define la creación del botón secundario o de cancelación
  Widget createSecondaryButton(
    String text,
    VoidCallback onPressed,
  );
}
```

```
// IMPLEMENTACION CONCRETA DEL ABSTRACT FACTORY
class MaterialFormFactory implements IAppUIFactory {

  @override
  Widget createInput(
    TextEditingController controller,
    String label, {
    String? hint,
    TextInputType? keyboardType,
    int maxLines = 1,
  }) {
    return TextField(
      controller: controller,
      keyboardType: keyboardType,
      maxLines: maxLines,
      decoration: InputDecoration(
        labelText: label,
        hintText: hint,
        border: const OutlineInputBorder(),
      ),
    );
  }

  @override
  Widget createPrimaryButton(BuildContext context, String text,
    VoidCallback onPressed) {
    return ElevatedButton(
      onPressed: onPressed,
      style: ElevatedButton.styleFrom(
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      child: Text(text),
    );
  }

  @override
  Widget createSecondaryButton(String text, VoidCallback
    onPressed) {
    return TextButton(
      onPressed: onPressed,
      child: Text(text),
    );
  }
}
```

4.1.4. Implementación de Builder

A continuación, se presenta la implementación del patrón Builder en Dart.

```
lib/builders/telefono builder.dart
```

```
class TelefonoBuilder {
  String _modelo = '';
  String _imagenUrl = '';
  String _descripcion = '';
  String _procesador = '';
  String _pantalla = '';
  int _numeroCamaras = 1;
  int _bateriaMah = 0;
  String _fechaSalida = '';
  Marca? _marca;

  // Método final que ensambla y devuelve el objeto resultante.
  // También puede incluir lógica de validación final.
  Telefono build() {
    if (_marca == null) {
      throw Exception('El teléfono debe tener una marca asignada.');
```

```
    }

    return Telefono(
      modelo: _modelo,
      imagenUrl: _imagenUrl,
      descripcion: _descripcion,
      procesador: _procesador,
      pantalla: _pantalla,
      numeroCamaras: _numeroCamaras,
      bateriaMah: _bateriaMah,
      fechaSalida: _fechaSalida,
      marca: _marca!,
      conteoReacciones: {}, // Estado inicial limpio dictado
                             por la arquitectura
    );
  }
}
```

4.1.5. Implementación de Prototype

A continuación, se presenta la implementación del patrón Prototype en Dart.

lib/models/telefono.dart

```
// PATRON DE DISEÑO CREACIONAL: PROTOTYPE
Telefono copyWith({
  int? id,
  String? modelo,
  String? imageUrl,
  String? descripcion,
  String? procesador,
  String? pantalla,
  int? numeroCamaras,
  int? bateriaMah,
  String? fechaSalida,
  Marca? marca,
  String? createdBy,
  Map<String, int>? conteoReacciones,
}) {
  return Telefono(
    id: id ?? this.id,
    modelo: modelo ?? this.modelo,
    imageUrl: imageUrl ?? this.imageUrl,
    descripcion: descripcion ?? this.descripcion,
    procesador: procesador ?? this.procesador,
    pantalla: pantalla ?? this.pantalla,
    numeroCamaras: numeroCamaras ?? this.numeroCamaras,
    bateriaMah: bateriaMah ?? this.bateriaMah,
    fechaSalida: fechaSalida ?? this.fechaSalida,
    marca: marca ?? this.marca,
    createdBy: createdBy ?? this.createdBy,
    // Ojo aquí: Si recibimos un mapa nuevo, lo usamos. Si no
    // , clonamos el actual
    // para evitar que ambos objetos compartan la misma
    // referencia en memoria.
    conteoReacciones: conteoReacciones ?? Map<String, int>.
      from(this.conteoReacciones),
  );
}
```


4.2. Implementación de Patrones Estructurales

4.2.1. Implementación de Adapter

A continuación, se presenta la implementación del patrón Adapter en Dart.

lib/adapters/date adapter.dart

```
// PATRON DE DISEÑO ESTRUCTURAL: ADAPTER
// Interfaz que define el contrato que el cliente (la UI)
// espera.
abstract class IDateAdapter {
  String getFriendlyDate(String rawDate);
}

/// Implementación concreta del Adaptador.
class FriendlyDateAdapter implements IDateAdapter {
  // Array auxiliar para no depender de librerías externas de
  // internacionalización
  static const List<String> _meses = [
    'enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio',
    'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', '
    diciembre'
  ];

  @override
  String getFriendlyDate(String rawDate) {
    if (rawDate.isEmpty) return 'Fecha desconocida';

    try {
      // 1. Recibimos la cadena "incompatible" y la parseamos a
      // un objeto DateTime
      final DateTime date = DateTime.parse(rawDate);

      // 2. Adaptamos la información al formato amigable
      // esperado por la UI
      return '${date.day} de ${_meses[date.month - 1]} de ${
        date.year}';
    } catch (e) {
      // 3. Fallback de seguridad por si el dato crudo no es
      // una fecha válida
      return rawDate;
    }
  }
}
```

4.2.2. Implementación de Bridge

A continuación, se presenta la implementación del patrón Bridge en Dart.

```
lib/bridges/app catalog bridge.dart
```

```
// PATRON DE DISEÑO ESTRUCTURAL: BRIDGE
class AppCatalogBridge {
  // El "Puente": una referencia a la interfaz de la
  // implementación, no a una clase concreta.
  final IApiRepository _apiImplementor;

  // Inyección de dependencias a través del constructor
  AppCatalogBridge(this._apiImplementor);

  // OPERACIONES DE ALTO NIVEL PARA LA UI
  // Aquí podemos agregar lógica de negocio antes o después de
  // delegar,
  // manteniendo la UI ignorante de cómo se obtienen los datos
  // realmente.

  Future<List<Marca>> loadAllBrands() async {
    // Delegación directa a la implementación
    return await _apiImplementor.fetchMarcas();
  }

  Future<List<Telefono>> loadAllPhones() async {
    return await _apiImplementor.fetchTelefonos();
  }

  Future<void> registerNewBrand(Marca marca) async {
    await _apiImplementor.createMarca(marca);
  }

  Future<void> registerNewPhone(Telefono telefono) async {
    await _apiImplementor.createTelefono(telefono);
  }

  Future<void> removePhoneFromCatalog(int id) async {
    await _apiImplementor.deleteTelefono(id);
  }
}
```

4.2.3. Implementación de Decorator

A continuación, se presenta la implementación del patrón Decorator en Dart.

```
lib/decorators/api_logger_decorator.dart
```

```
class ApiLoggerDecorator implements IApiRepository {
  // El decorador debe contener una referencia al objeto
  // original que está envolviendo
  final IApiRepository _innerRepository;

  ApiLoggerDecorator(this._innerRepository);

  // Método auxiliar privado para no repetir la lógica del cron
  // ómetro
  Future<T> _logExecutionTime<T>(String methodName, Future<T>
    Function() action) async {
    print('[API LOGGER] Iniciando petición: $methodName...');
    final stopwatch = Stopwatch()..start();

    try {
      // Delegamos la acción real al objeto envuelto
      final result = await action();
      stopwatch.stop();
      print('[API LOGGER] Exito en $methodName. Tiempo: ${
        stopwatch.elapsedMilliseconds} ms.');
```

4.2.4. Implementación de Facade

A continuación, se presenta la implementación del patrón Facade en Dart.

```
lib/facades/dashboard facade.dart
```

```
// PATRON DE DISEÑO ESTRUCTURAL: FACADE
class DashboardFacade {
  // La fachada interactúa con los subsistemas complejos (en
  // este caso, el puente)
  final AppCatalogBridge _catalog;

  DashboardFacade(this._catalog);

  // Interfaz simplificada para la UI.
  // La vista solo llama a esto y no le importa cómo se
  // consigue la información.
  Future<DashboardData> getFullDashboard() async {
    try {
      // Optimizamos: Usamos Future.wait para pedir ambas
      // listas AL MISMO TIEMPO
      // en lugar de esperar a que termine una para empezar la
      // otra.
      final resultados = await Future.wait([
        _catalog.loadAllBrands(),
        _catalog.loadAllPhones(),
      ]);

      // Extraemos los resultados tipados
      final marcas = resultados[0] as List<Marca>;
      final telefonos = resultados[1] as List<Telefono>;

      // Empaquetamos y entregamos a la UI
      return DashboardData(marcas, telefonos);
    } catch (e) {
      // Manejo de errores centralizado del subsistema
      throw Exception('Fallo en el subsistema del Dashboard: $e');
    }
  }
}
```

4.2.5. Implementación de Flyweight

A continuación, se presenta la implementación del patrón Flyweight en Dart.

```
lib/models/reaccion_tipo.dart
```

```
// PATRON DE DISEÑO ESTRUCTURAL: FLYWEIGHT
// Propósito: Minimizar el uso de memoria RAM compartiendo la
// mayor
// cantidad de datos posibles con objetos similares. En Dart,
// los 'enum'
// son una implementación perfecta de este patrón, ya que
// garantizan que
// las instancias sean constantes en tiempo de compilación y
// compartidas
// globalmente a lo largo de toda la aplicación.

enum ReaccionTipo {
  like(1, '', 'REACCION_LIKE'),
  corazon(2, '', 'REACCION_CORAZON'),
  risa(3, '', 'REACCION_RISA');

  // Datos inmutables que componen el estado intrínseco
  // compartido
  final int id;
  final String emoji;
  final String nombreDb;

  const ReaccionTipo(this.id, this.emoji, this.nombreDb);

  // 2. Método de fábrica (Factory Method) encapsulado dentro
  // del Flyweight.
  // Permite recuperar la instancia compartida correcta a
  // partir de un dato externo.
  static ReaccionTipo fromNombreDb(String nombre) {
    return ReaccionTipo.values.firstWhere(
      (r) => r.nombreDb == nombre,
      orElse: () => ReaccionTipo.like, // Fallback seguro
    );
  }
}
```

4.2.6. Implementación de Proxy

A continuación, se presenta la implementación del patrón Proxy en Dart.

lib/proxies/api_auth_proxy.dart

```
// PATRON DE DISEÑO ESTRUCTURAL: PROXY (Protection Proxy)
class ApiAuthProxy implements IApiRepository {
  // Referencia al objeto real que hace el trabajo pesado
  final IApiRepository _realRepository;

  ApiAuthProxy(this._realRepository);

  // Lógica interna del proxy para verificar los "pases de
  // acceso"
  Future<bool> _isUserAuthenticated() async {
    final prefs = await SharedPreferences.getInstance();
    final token = prefs.getString('jwt_token');
    return token != null && token.isNotEmpty;
  }

  // MeTODOS PROTEGIDOS (De escritura):
  // El proxy intercepta y bloquea si el usuario no tiene
  // credenciales.
  @override
  Future<Telefono> createTelefono(Telefono telefono) async {
    if (!await _isUserAuthenticated()) {
      throw Exception('PROXY INTERCEPTO: Acceso denegado.
        Inicia sesión para publicar teléfonos.');
```

4.3. Resultados

Una vez que hemos integrado todos los patrones de diseño designándoles una única responsabilidad, podemos armar nuestro archivo *main* con un nivel de acoplamiento tan bajo que no tiene nada que envidiar a los códigos profesionales de alta calidad.

Una vez todos los patrones estén funcionando, bastará con mandar a llamar sus respectivos métodos a través de abstracciones, dejando el código abierto a la extensión a futuro. Por ejemplo, si en algún momento del ciclo de vida del proyecto se cambia el host o motor de base de datos del proyecto, bastaría con crear una nueva implementación del patrón *factory method* y crear su instancia desde el archivo *main*. De esta forma, el solamente tendríamos que hacer una modificación mínima dentro de la interfaz del usuario, demostrando la calidad del código a través del bajo acoplamiento de módulos de alto nivel con los de bajo nivel y viceversa.

Por otro lado, implementar estos patrones de diseño resultan en una experiencia de usuario sumamente agradable, por lo que, la inclusión de estos patrones fue un cambio total en la madurez del proyecto.

5. Conclusiones

A través de este experimento de implementación de patrones de diseño en un proyecto de software, podemos decir que, conocer y dominar los patrones de diseño creacionales y estructurales nos puede ayudar en nuestra formación tanto académica como en nuestro desempeño laboral en un futuro. La inclusión de patrones de diseño en proyectos de software demuestran dominio sobre lo que es la calidad de software, debido a que, no solamente nos estaremos centrando en entregar lo que un cliente necesita, sino que, estaremos entregando un producto de calidad bajo estándares de la industria, cumpliendo con los tres grandes atributos de calidad de un código **leíble, mantenible y testeable**.

Si bien, no es estrictamente necesario incluir absolutamente todos los patrones de diseño en un proyecto (como bien se demostró en este proyecto en referencia al patrón *composite*), tomar en cuenta la inclusión de dichas soluciones puede garantizar una madurez profesional en el desarrollo de un sistema, a la vez que, podría ayudara exponencialmente en la calidad del sistema en términos de sus características como eficiencia, confiabilidad, rapidez, etc.

Finalmente, con este proyecto se demuestra la gran adaptabilidad de los patrones de diseño, lo que demuestra la razón de por qué son un estándar a nivel industrial. La adaptabilidad en patrones como el *Facade*, demuestran que estas soluciones van más allá de una solución a un problema muy específico, puesto que los patrones son soluciones que en gran parte de proyectos de software los programadores se pueden enfrentar, siendo altamente adaptables a distintos tipos de proyectos.